# INDIVIDUAL TECHNICAL REPORT

*COMP713 Assessment 3 | Fieldwork - Distributed and Mobile System*

Shirui Zang | 21136179 | Solo project | 31 October 2026

Source repository: https://github.com/SaltedFishInATank/COMP713-Assignment3-ShiruiZang

Published branch checked: main, 3ac79d702349c9ae3a6e54c188257b1e9a99bce9 | Original local build label: assessment3-local-v3.0.0

STUDENT REVIEW COPY - This document provides an evidence-grounded technical narrative, not a claim of independent authorship. The assignment requires the individual report to be independently written. Review and rewrite the reflective passages using your actual decisions, commands, experience and video before submitting. Do not present an unreviewed AI-prepared version as independent work.

## 1. System understanding

Fieldwork extends the Assessment 2 Project and Task Manager to support a two-role campus maintenance workflow. A coordinator uses the browser to create projects, assign tasks and review submissions. A field worker uses the native Android client to retrieve assigned work, accept it, and submit completion notes with location evidence. Both clients are linked by the same HTTP/JSON API hosted as a Jakarta EE WAR on Payara Server 6. This means neither client owns business state: the Payara backend and its H2 database are authoritative.

A typical first workflow begins when the coordinator creates a task in the web application. Jakarta REST resources in backend/src/main/java/nz/ac/aut/comp713/api/TaskResource.java receive the request. AuthService and WorkflowService apply identity, role and workflow rules, while WorkflowRepository and Database use prepared SQL and explicit JDBC transactions. The task is saved with an assigned worker and a version number. When the worker signs in through android/app/src/main/java/nz/ac/aut/comp713/fieldtasks/MainActivity.java, ApiClient.java retrieves only the permitted assignments. Accepting a task changes its persisted state to in_progress. Another worker cannot accept that task under the intended authorization rules.

The second workflow covers inspection evidence and review. The Android app obtains a location using Android device services when permission and a valid fix are available, but accepts manual coordinates as a fallback. The worker submits a completion note and position through the Payara API. The service validates the inputs and task version and changes the state to awaiting review. The coordinator can then approve the work or request rework. A stale client version produces HTTP 409, preventing a late write from silently overwriting a more recent decision. Android can retain the draft while retrieving the latest version.

The live-update path is separate from data persistence. After a relevant database commit, TaskSocket.java emits a WebSocket task.changed notification to the open browser. That notification tells the browser to request the current API state rather than treating the socket message as a replacement database. After WebSocket reconnection, a fresh REST read restores the latest state. This distinction matters when a client has missed one or more events during an outage.

The selected advanced integration is Path B: a message broker. Task submission and a stable-ID task.submitted outbox event are saved in one backend transaction. TaskEventPublisher.java later sends pending events through Payara's OpenMQ broker. The separately running Java consumer, implemented in NotificationConsumer.java and ConsumerStore.java, owns a different H2 notification store. It records the event effect before acknowledging delivery, so a redelivered event can be identified by its event ID and does not create a duplicate notification. This is at-least-once delivery with idempotent processing, not exactly-once message transport.

## 2. Critical reflection and technical issue analysis

### 2.1 Architecture trade-off

The implemented backend retains the JDBC/service/repository separation from Assessment 2 rather than replacing it with JPA. This reduces the number of changes needed to extend familiar Project and Task operations and makes SQL transactions explicit. It also leaves more manual record mapping and connection management, and it does not use a Payara-managed JDBC pool. For a small coursework system, that is an understandable trade-off; for a larger deployment, a managed datasource and clearer migration tooling would reduce operational risk. This assessment is based on the source architecture and recorded behaviour, not an independently performed comparison benchmark.

### 2.2 Concrete technical issue: Payara Tools registration

During preparation of the demonstration, I attempted to register Payara Server in my regular VS Code window. The file picker initially showed the Assessment3/backend directory containing .vscode, src and target, rather than the Payara installation. I then reported that VS Code would not register the server. This is a reproducibility problem in my local workflow, not proof that the backend application itself is defective. The supplied automated records separately show successful Payara Tools Start and Run on Payara Server commands in a configured isolated VS Code extension profile. Server registration was preseeded in that test profile, so the automated success does not establish that my ordinary VS Code registration wizard was working.

The likely distinction is between the application folder, the Payara Server installation directory and the domain folder. The correct server home is expected to contain bin and glassfish, with domain1 under glassfish/domains. I cannot state that I resolved the manual registration issue because no successful normal-profile registration was evidenced in the supplied materials. The project includes scripts/Payara.ps1 as a CLI fallback to start, deploy and inspect the same backend. A useful next verification is to register the installed Payara directory in the normal VS Code profile, start domain1, deploy backend/target/comp713-a3.war, and confirm HTTP 200 at /comp713-a3/api/health while capturing the actual extension output.

### 2.3 Testing result and diagnosis

The preserved performance logs document a measurable issue discovered during tool-assisted verification. A warm authenticated task-detail GET initially missed the selected p95 <= 500 ms target, with p95 approximately 699.81 ms. Investigation attributed the delay to repeated H2 file

shutdown and reopening after the last JDBC connection closed. The revised source uses DB_CLOSE_DELAY=10 and an application shutdown hook. A later local run of 40 sequential requests reported p95 39.31 ms (minimum 20.66 ms, maximum 40.70 ms). These measurements support the correction under the recorded conditions, but they are limited to localhost, one client and a warm database; they do not prove performance under concurrent mobile traffic.

The test evidence also reports nine passing backend JUnit tests, three passing consumer JUnit tests, 15 live API/WebSocket checks, 11 Chrome checks, and Android 14 emulator runs that covered a failed login, assignment acceptance, location denial/manual fallback, a stale-version 409 and approval refresh. These are results preserved by the Codex-assisted verification process. The documentation should not imply that I personally reran each test unless I have done so and can provide that evidence.

#### 2.4 Reliability, limitations and improvement priority

A useful reliability property of the outbox is that task state and event intent are committed together; a temporary broker outage does not simply erase the submission event. The consumer must handle delivery more than once, because processing can complete before acknowledgement is confirmed. The recorded retry, replay and post-save/pre-ack tests show why a stable event ID and unique persistent effect are needed. I would prioritise making local setup reproducible, followed by testing the Android workflow on a physical phone and a non-local network. A further production-oriented improvement would be to use HTTPS, managed connection pooling and application monitoring rather than relying on one local Payara and H2 instance.

## 3. Personal contribution and evidence

This was organised as a solo coursework project. I supplied the Assessment 2 baseline and class materials, specified that the system should use Payara Tools/Payara Server, and used Codex to help extend the original application into web and Android clients with shared persistence, WebSocket updates and a broker consumer. I created and published the source repository using my GitHub account. The published main branch records the project import at 5a097e32 and a repository-information update at 3ac79d70, both dated 31 October 2026. These commits establish repository publication and a later documentation change, not three weeks of personal coding or manual authorship of the generated source.

My directly observed manual activity includes opening the project in VS Code, attempting Payara Server registration, and preparing a demonstration recording. The student has reported that a video is now available, but this report draft cannot validate its duration, audio/subtitles or which failure scenarios it shows because the video has not been supplied here. The current source and automated test logs show substantial technical functionality, while the role and extent of my own manual implementation, review and retesting should be stated only where I can support them with actual observations or file changes.

Assessment 2 already contained the basic Project/Task domain, Jakarta REST registration, JDBC repository and browser patterns; Assessment 3 added role-aware assignment and review, optimistic versions, Android location-supported submissions, WebSocket restoration, and

OpenMQ outbox/consumer processing. The principal evidence paths include backend/src/main/java/nz/ac/aut/comp713/service/WorkflowService.java, backend/src/main/java/nz/ac/aut/comp713/websocket/TaskSocket.java, android/app/src/main/java/nz/ac/aut/comp713/fieldtasks/MainActivity.java, consumer/src/main/java/nz/ac/aut/comp713/consumer/NotificationConsumer.java, evidence/live-api.json, evidence/web/result.json and android/evidence/verification.json. My report should link only the files and tests I genuinely reviewed.

The source evidence contains two dated implementation checkpoints on 31 October 2026: an Android-to-API/browser update at approximately 13:23 NZDT and a later Android GPS submission reaching OpenMQ and the independent consumer at approximately 13:24. The published repository was updated on the same day. This is useful traceability but does not meet the assessment requirement for meaningful commits spanning at least three weeks. I will not backdate contributions or claim that the one-day history satisfies that criterion.

AI assistance disclosure. Codex assisted with code generation, integration, automated verification, documentation and initial report drafting. I remain responsible for explaining the architecture, checking evidence against the running system, identifying limitations and writing the final individual reflection in my own words. The group requirement specifies 2-3 students; the solo arrangement should be confirmed with the course coordinator rather than implied to have been approved.

## Evidence and version reference

Assignment: Assessment 3 specification, pp. 1-4. Source repository: https://github.com/SaltedFishInATank/COMP713-Assignment3-ShiruiZang. Latest public main commit observed: 3ac79d702349c9ae3a6e54c188257b1e9a99bce9. Local code archive used for documented execution: assessment3-local-v3.0.0. Backend tests: evidence/maven-verify-final.log and evidence/live-api.json. Browser: evidence/web/result.json. Android: android/evidence/verification.json and native-runtime.log. Broker: evidence/broker/ and consumer evidence. Working milestones: docs/progress.md. These references should be checked against the final submitted ZIP, not assumed identical to later GitHub changes.